ReproBot: A Multi-Agent System for Automated Scientific Paper Replication

Final Project Report

Aleyna Kütük
aleynakutuk10@gmail.com

Engincan Varan
engincanvaran@gmail.com

Gül Korkut
serifegulkorkut@gmail.com

Göktuğ Mert Özdoğan
goekmeroz@gmail.com

inzva AI Projects #10
10.10.2026

Abstract

ReproBot reads a machine learning paper's PDF and returns a replication report. It transcribes the paper with a vision-language model, extracts its claims and experimental setup into structured fields, generates a self-contained training script, executes that script in a Docker sandbox while watching the learning curve for signs that training is not healthy, judges the reproduced number against the paper's claim with explicit arithmetic, reviews the generated code against the paper, and renders the result as Markdown. Failures at either end — a script that crashes and a number that misses — are fed back to the code generator through one bounded retry loop. This report presents the finished system and everything it has measured. Five papers spanning five model families were reproduced at their own full settings and all five passed, two of them beating the paper's own number. Three of those papers were then carried from PDF to report three times each: 9 of 9 runs reproduced their claim, and every run reached the full stage on its first attempt. The central finding is about where the remaining disagreement comes from: every difference between runs of the same paper traced to exactly one detail that paper never states, and we could name it each time — a fully specified protocol gave three independently generated scripts bit-identical values, while an unstated cross-validation shuffle and an unstated data split each moved the result in a way we can explain. A loop-on/loop-off ablation shows the retry loop changed 1 of 5 verdicts, and that the system's real defence against publishing a wrong number is not the retries but the guards: a live check-up that halts an unhealthy run, and a Critic that refuses to rule on one. We make no fidelity claim for the eight-paper CIFAR-10 set: a full *Wide Residual Networks* run measures at about 22 days on the project's CPU, and a one-epoch run proves that a script executes, never that its number is right.

1 Introduction

Most published machine learning results are reproducible in principle and expensive to reproduce in practice. The method is in the prose, the numbers are in the tables, and the thousand small decisions in between — which split, which initialisation, which library version's defaults — are in neither. ReproBot automates the whole path: paper in, replication report out, with every guess it had to make written down.

The system is a pipeline of seven stages and two bounded retry loops, coordinated through one shared-memory state object. It is complete: every stage in the project plan exists, has run on real papers, and is verified by execution rather than by inspection. This report is self-contained — it assumes no earlier reading — and leads with results, because the results are the point.

Its contributions are:

- **Five full-fidelity replications across five model families**, each judged by arithmetic rather than by eye, with two reproductions better than the paper's own number (Section 3).
- **A repeatability measurement**: three papers carried from PDF to report three times each, 9 of 9 passing (Section 3.2).
- **The report's main finding**: run-to-run variance is a property of the paper, not a defect of the system, and in every case we could name the single unstated detail responsible (Section 3.3).
- **An explicit, model-proof verdict**: the tolerance is arithmetic over the claim's own precision and the run's own measured spread, and no model can change it (Section 4).
- **The project plan's headline ablation**, loop on against loop off, reported honestly including where the loop bought nothing (Section 5).
- **Two documented paper sets**, one CPU-sized and one GPU-bound, and a plain statement of which results exist for which (Section 6).

2 The System

Figure 1 shows the whole system; Table 1 says what each stage does, what it spends a model call on, and what it checks without one. The design rule throughout is that anything decidable by arithmetic is decided by arithmetic: a model is called where judgement is genuinely needed — reading a figure, writing code, diagnosing a traceback, reviewing an implementation — and nowhere else.

The loop. Two feedback edges close it. On an execution failure, a small model classifies the failure as a bug in the generated script or a problem with the container; a script bug is routed back to the Coder as a specific complaint and the script is regenerated. On a fidelity failure, the Critic's verified findings and the Coder's own list of unstated guesses become the feedback for one guided retry. Both edges are bounded: a retry budget, a single fidelity retry per paper, and a plateau guard that compares each regenerated script with its predecessor and stops early when the "fix" is not changing anything.

The parts that must not be wrong are tested. The pure logic — the Critic's arithmetic, the loop's four decision functions, the seven check-up rules and the report renderer — is covered by 102 replay tests, built from real recorded learning curves rather than synthetic shapes, including every false positive found end to end.

What the stages run on. The Runner's interface is `reproduce.sh <mode>` and nothing else — it never builds a `python` command and never passes a flag — so a paper whose script needs entirely different arguments changes only its own `reproduce.sh`. The four modes escalate: `probe`, a few optimiser steps; `smoke`, one epoch on a small slice; `capped`, a few epochs on a small slice; and `full`, the paper's own setup. The Critic may additionally ask for `seed2` and `seed3`, which are `full` again with another seed.

3 Results

3.1 Fidelity: five papers, five model families

Table 2 is the headline result. Five papers were reproduced at their own full settings — not a reduced proxy — and judged by the arithmetic of Section 4. All five passed. Two reproductions

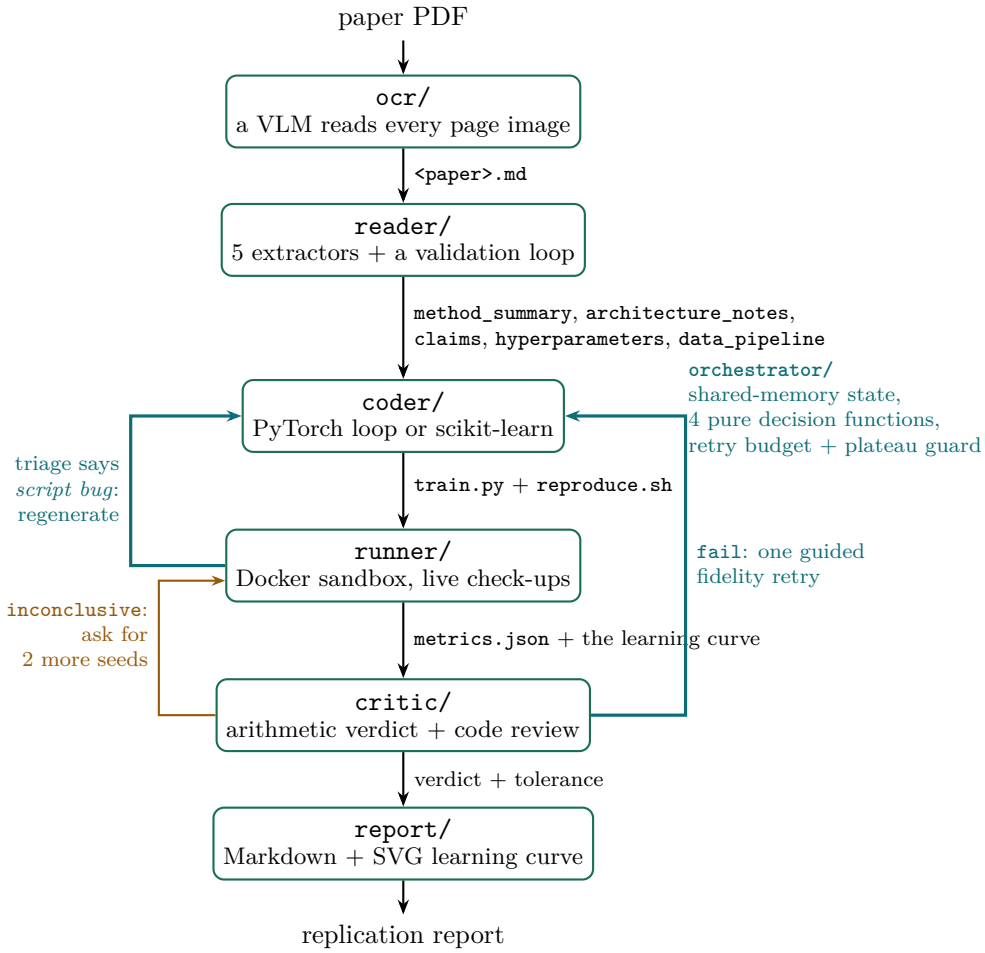


Figure 1: ReproBot, complete. Every stage is built and verified by execution. Two feedback edges close the loop: the Runner's triage routes a script bug back to the Coder, and the Critic routes a fidelity failure back as one guided retry, or asks the Runner for extra seeds instead of ruling. Both are bounded by the Orchestrator's budgets and by a plateau guard that stops a retry which is not changing the script.

are better than the paper's own number by more than the tolerance band, which the Critic records as `exceeds_claim`. Figure 2 shows four of the implementations learning.

Two of the five needed a human correction before their final run, and the report says which: Tang's unstated SVM penalty was changed from the common default $C = 1.0$ to $C = 0.1$ after a six-configuration ablation, and the soft-tree paper's misprinted loss was implemented as its evident intent. Both are in Table 14. The other three ran unassisted.

Tang: an MLP with a margin loss. The network trained for the paper's own 400 epochs, training error falling to 0.062% (Figure 2a), and the test set — evaluated once, at the end — gave 0.82% against the paper's 0.87%: five test images out of 10,000, well inside the ±0.1857 that binomial noise on a 10,000-image test set allows.

The SVM guide: the cleanest exactness test available. Because scikit-learn's `SVC` wraps the same LIBSVM the guide itself used, the library *is* the method, and at the paper's own settings the reproduction matches its reported accuracies exactly (Table 3). Everything that remains is model selection: Figure 2b shows the cross-validation surface the generated script searched, with a broad ridge of settings above 96% and the paper's own choice one cell away from the search's.

Fashion-MNIST: a claim that is already an average. The paper reports the mean of

Table 1: The seven stages. Each is a top-level folder with the same command-line shape (`--input`, `--output`, skip what is already done), so any stage can be run alone or as part of the loop.

Stage	What it does	Model calls	Checked without a model
`ocr/`	Renders every PDF page to an image and transcribes it, reading figures as well as text	One per page	—
`reader/`	Five structured extractions — method summary, architecture notes, claims, hyperparameters, data pipeline — then a validation pass that re-runs only the stage a flag belongs to	Six, all forced structured output	Bounded at three validation passes; truncation, empty and double-encoded payloads caught centrally
`coder/`	One self-contained training script: a plain PyTorch loop for neural models, a scikit-learn estimator for classical ones	One	`ast.parse`; all nine required command-line flags present; the learning-curve progress contract
`runner/`	Runs `reproduce.sh <mode>` in Docker over four escalating stages, stopping at the first failure	One triage call, and only on a genuine non-timeout failure	Seven live check-up rules that can kill the container; per-stage timeouts; success and timeout decided mechanically
`orchestrator/`	Holds the shared-memory state and drives the retry loop as four pure decision functions	None	Plateau guard on regenerated scripts; retry and seed budgets
`critic/`	Judges the reproduced number against the claim, and reviews the code against the paper	One review call	The verdict itself: arithmetic only. Every citation in the review is checked to exist
`report/`	Renders the state object as Markdown with an SVG learning curve	None	Every number copied from the state; the tolerance printed with every verdict

five runs with shuffled training data, so the generated script performs five runs and reports all five: 0.8773, 0.8774, 0.8753, 0.8792 and 0.8775, mean 0.87734 against the claimed 0.873. That measured spread is what gives this row the tightest tolerance in the table, ±0.00175 — and the reproduction still clears it on the high side. Figure 2c shows why 100 trees was a safe choice: accuracy is already on its plateau by 30.

The soft decision tree: an under-specified paper reproduced through disclosed guesses. The paper states the tree's depth and essentially nothing else about training, so the generated script chose and recorded a value for each missing setting. Test accuracy passed the paper's 94.45% at epoch 11 and finished at 95.11% (Figure 2d). No setting was chosen on the test set, so a result 0.7 points above the claim with every guess disclosed is a comfortable reproduction rather than a lucky one.

Wijaya: the one that needed more than one run. The first full run of the housing-price network gave test RMSE 3.1285 against the claimed 3.02 — worse than the claim, with no noise model to say whether that mattered. The Critic returned `inconclusive` and asked

Table 2: Reproduced results against the published claims, every one at the `full` stage. ↑ marks a result better than the paper's own beyond the tolerance band. The last row is deliberate: the CIFAR-10 set has no fidelity verdict at all.

Paper	Model family	Dataset	Paper claims	ReproBot	Verdict	Tolerance
Tang (2013) [8]	MLP with an L2-SVM loss (PyTorch)	MNIST, 60k/10k	0.87% test error	**0.82%**	`pass`	±0.1857
Hsu, Chang & Lin (2003) [3]	RBF SVM (scikit-learn `SVC`, i.e. LIBSVM [4])	svmguide1, 3,089/4,000, 4 features	96.9% accuracy	**96.625 – 96.925%** over 3 runs	`pass` 3/3	±0.5481
Xiao et al. (2017) [19]	Random forest (scikit-learn [7])	Fashion-MNIST, 60k/10k, raw pixels	0.873 accuracy, mean of 5	**0.87734** ↑	`pass`	±0.00175
Frosst & Hinton (2017) [20]	Soft decision tree (PyTorch)	MNIST	94.45% accuracy	**95.11%** ↑	`pass`	±0.4579
Wijaya (2023) [26]	Dense regression network (paper used Keras)	Boston Housing [6], 405/101, 13 features	test RMSE 3.02	**3.03629**, mean of 3 seeds	`pass`	±0.2044
CIFAR-10 set, 8 papers	Image CNNs and transformers	CIFAR-10	—	executes to `smoke`	`not_evaluated`	needs a GPU

Table 3: svmguide1 test accuracy. The paper's three reported settings are reproduced exactly by scikit-learn's `SVC`, because it wraps the same LIBSVM implementation. Only the grid search's choice differs. Paper values read from the PDF itself.

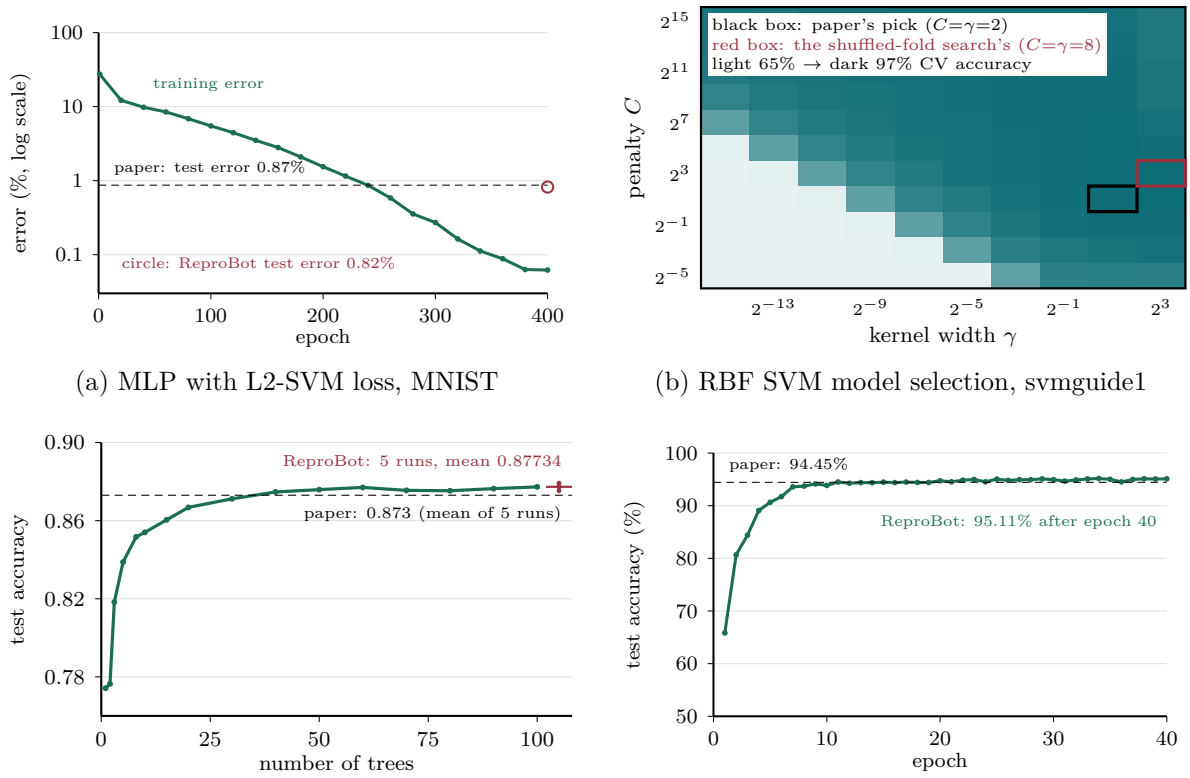
Setting	Paper	ReproBot
Unscaled features, library defaults	66.925%	66.925%
Scaled to $[-1, 1]$, library defaults	96.15%	96.15%
Scaled, $C = 2$, $\gamma = 2$	96.875%	96.875%
Scaled, the paper's full grid-search procedure	96.9%	96.625 – 96.925%

for seeds instead of ruling. Two more full runs gave 3.0283 and 2.9520, and the re-judgement was `pass` at a mean of 3.03629 with a tolerance of ±0.2044 that was now measured rather than assumed. A system that ruled on one number would have called this paper a failure at 3.13, or a success at 2.76, depending only on which run you happened to see.

3.2 Repeatability: three papers, three runs each, PDF to report

A single successful run proves little, so three of the papers were run three times each, from the PDF through to a rendered report, with the whole loop switched on: retries, live check-ups, the Critic's verdict, its seed escalation and its guided fidelity retry. Each paper was re-read from its PDF the same day, so no extraction was inherited from an earlier session.

9 of 9 runs reproduced their claim, and every run reached `full` on its first attempt — no retry was consumed. End-to-end wall clock was 77 to 238 seconds per run, so



(a) MLP with L2-SVM loss, MNIST

(b) RBF SVM model selection, svmguide1

(c) Random forest, Fashion-MNIST

(d) Soft decision tree, MNIST

Figure 2: Four of the five generated implementations at the papers' own full settings, with each paper's reported result as a dashed line. (a) Training error per 20 epochs, log scale; the circle is the reproduced test error, evaluated once at the end. (b) Five-fold cross-validation accuracy over the SVM's (C, γ) grid, with the paper's own setting and the shuffled-fold search's pick outlined. (c) Test accuracy as trees are added, for the first of five repetitions, with all five repetitions' final accuracies at the right. (d) Test accuracy per epoch. Panels (b) and (c) were computed with the generated scripts' own configuration and code; all other points are read directly from the run logs.

the whole batch cost about 35 minutes of laptop CPU.

3.3 Run-to-run variance is a property of the paper

This is the finding we would keep if we could keep only one. Across the nine runs, **every disagreement between two runs of the same paper traced to exactly one detail that paper never states, and in every case we could name it.** Table 5 gives the mechanism for each, and Figure 3 plots every reproduced value this project holds for the two papers it ran most often.

The SVM guide's mechanism is worth stating exactly, because it is counter-intuitive. The grid search optimises cross-validation accuracy, and here the cross-validation winner was the test loser: the shuffled-fold run's choice won cross-validation by 0.19 points and lost on test by 0.30. Both runs pass, and both are faithful implementations of what the paper describes. The paper's own answer, $C = \gamma = 2$, gives 96.9%. One unstated word — shuffled — is the whole difference.

Fashion-MNIST's is worth stating because it is the control. Three generations whose texts barely resemble each other produced the same five numbers to four decimal places. When a paper specifies its protocol completely, ReproBot's output is deterministic across generations. That is the strongest available evidence that the variance seen elsewhere is not the

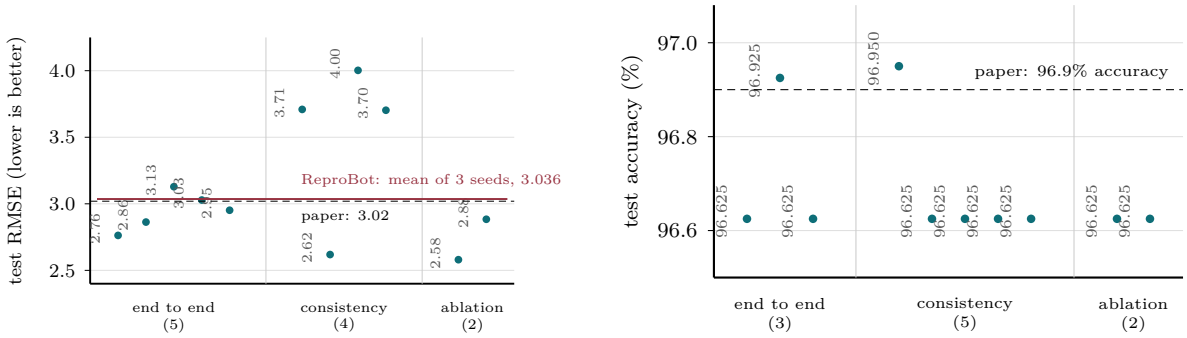
Table 4: Three papers, three independent runs each, 2026-10-09. The claim is pinned by id, because target-claim selection is not reproducible across Reader runs. The last column is the finding: each paper's spread has exactly one named cause.

Paper	Claim	Passes	Run 1	Run 2	Run 3	Why the runs differ
SVM guide (`c2`)	96.9%	**3/3**	96.625	96.925	96.625	The paper says "five-fold cross-validation" and never says whether the folds are shuffled
Fashion-MNIST (`c41`)	0.873	**3/3**	0.87734	0.87734	0.87734	Nothing — the protocol is fully specified
Wijaya (`c3`)	RMSE 3.02	**3/3**	2.76299	2.86305	3.03629*	The paper gives the 405/101 split sizes and never says how rows are assigned

* Run 3 is the mean of three seeds. Its first value, 3.1285, was judged `inconclusive` and the Critic escalated on its own.

Table 5: The mechanism behind each paper's spread. Variance here is not noise in the system; it is the paper's own under-specification, made visible and named.

Paper	What the paper does not state	What that changes	Observed consequence
Fashion-MNIST	Nothing relevant: official 60k/10k split, raw pixels, 100 trees, entropy criterion, depth 100, five repetitions with shuffled data	—	Three independently generated scripts of 303, 274 and 291 lines — 299 and 290 changed lines against the first — produced **bit-identical** values. All three independently chose base seed 42, per-repetition `seed + i`, and `random_state=rep_seed` on both the shuffle and the forest
SVM guide	Whether the five cross-validation folds are shuffled	Which (C, γ) the grid search selects, and therefore the test accuracy	Runs 1 and 3 shuffled with seed 42, selected C=8, γ=8 at CV 96.9894% and scored 96.625% — identically. Run 2 used scikit-learn's unshuffled contiguous folds, selected C=2, γ=8 at a *lower* CV of 96.7952%, and scored **96.925%**
Wijaya	How the 405/101 split is drawn	Which rows are held out of a 506-row dataset	The RMSE moves continuously: 2.76299, 2.86305, 3.03629 across three runs, and 2.619 to 4.003 across five independent generations



(a) Wijaya 2023: the split is never stated

(b) SVM guide: the fold shuffle is never stated

Figure 3: Every reproduced value this project holds for the two papers it ran most often, grouped by the experiment that produced it; the count in each group label is the number of runs. The horizontal axis carries no quantity — each run gets its own slot so nothing overlaps. (a) Wijaya never states how its 405/101 split is drawn, so every generation draws its own and the RMSE moves continuously; the red line is the judged mean of the three seeds the Critic escalated to. (b) The SVM guide never states whether its five cross-validation folds are shuffled; every run that shuffled them landed on 96.625%, and the two that did not landed above the claim. Both papers' claims sit inside their spread.

Table 6: Claims extracted against claims tested. Merging collapses one result printed in prose, a table and a figure; it only bites where a paper repeats itself. A dash means merging was never run for that paper.

Paper	Claims	Distinct	Tested
Fashion-MNIST (2017)	74	70	1
Deep Residual Learning (2015)	66	—	0
Wide Residual Networks (2016)	61	—	0
SVM guide (2003)	27	27	1
All Convolutional Net (2014)	17	—	0
Tang (2013)	15	15	1
Network In Network (2013)	12	—	0
Frosst & Hinton (2017)	8	8	1
Wijaya (2023)	4	4	1

Coder being unreliable.

3.4 Coverage: how much of each paper was tested

A replication that targets one claim is fast and partial, and the reports say so. Table 6 gives the denominator: how many claims the Reader extracted from each paper, how many distinct results remain after merging duplicates, and how many were actually reproduced. Every generated report lists the untested remainder explicitly as `not tested` rather than omitting it.

Claim counts are a property of a particular Reader run, not of the paper: an earlier extraction of Wijaya found 14 claims that merged to 8 distinct results, where the run in the table found 4. That instability is why every experiment in this report pins its target with `--claim-id` rather than letting the model choose again.

3.5 Does the code match the paper? The Critic's review

The verdict says whether the number matched; it cannot say whether the number was produced the way the paper describes. A second Critic call reads the Reader extraction, the paper, the

Table 7: Code review of the three end-to-end run-1 scripts, reviewer `claude-opus-5`. No review reported a deviation from the paper or an implementation bug.

Paper	Fidelity	Deviations	Citations
SVM guide	`faithful`	none	10 / 10
Fashion-MNIST	`faithful`	none	8 / 8
Wijaya	`faithful`	none	12 / 15

line-numbered script, the Coder's recorded assumptions, the learning curve and a filtered runner log, and returns findings — each one a claim about the code with a paper quote and script lines attached — plus ranked hypotheses for any gap. Deterministic guards then decide what reaches the Coder: a cited number must appear in the material, a paper quote must exist in the paper, script lines and snippets must exist in the script, and a proposed fix may not introduce a technique the paper never mentions. Findings that fail a guard are dropped, and the count of dropped findings is printed in the report.

Two things about Table 7 are worth more than the ratings themselves. First, the reviewer agreed with the arithmetic independently — it read the code, not the verdict. Second, the Wijaya review predicted its own result's direction without being told it: the unstated split assignment, it wrote, "can move test RMSE by several tenths, which plausibly explains the reproduced 2.763 landing below the claimed 3.02." That is the same mechanism Table 5 names, arrived at from the code.

The reviewer is `claude-opus-5` on evidence, not preference. On a Wijaya script with a deliberately swapped train/validation unpacking, Sonnet 5 missed the bug three times — even with the log stating the two split sizes — and misdiagnosed it as a missing early-stopping rule, which made the Coder add checkpoint selection that the paper never mentions. Opus found the swap, the loop fixed it, and the next review returned `faithful`: a fidelity `fail` at 4.76 became a `pass` at 3.40 against 3.02, in eleven minutes.

4 How a Verdict Is Made

The fidelity verdict is arithmetic. It makes no model call, opens no network connection and starts no container, which means no model can argue its way to a pass:

$$\text{tolerance} = \max(2u,\ p),$$

where p is the claim's own reporting precision and u the run's uncertainty. Precision is read from the claim as printed: a claim of 3.02 is precise to ± 0.005. Uncertainty comes from the run itself where the run can supply it — if two or more runs exist, from their measured spread, as $\sigma\sqrt{2/n}$ when the claim is itself a mean and $\sigma\sqrt{1 + 1/n}$ when a single-run claim is being compared with our mean — and otherwise, for an accuracy-like metric, from the binomial noise of a test set that size. When neither is available there is no uncertainty estimate, and the band collapses to the claim's own precision.

Four verdicts follow, and each carries a consequence:

- `pass` — within tolerance, or better than the claim beyond it, which is recorded separately as `exceeds_claim`.
- `fail` — worse than the claim beyond tolerance, *with* an uncertainty estimate. If the Critic's review found verified problems the paper itself states, this becomes a normal retry; otherwise it spends the single guided fidelity retry.

Table 8: Where each tolerance came from, and how wide it is relative to the claim. A tighter band is a stronger pass. The two Wijaya rows are the same paper judged on one run and on three.

Paper (claim)	Uncertainty source	Tolerance	of claim
Fashion-MNIST (0.873)	Measured spread of 5 runs	±0.00175	0.20%
SVM guide (96.9%)	Binomial noise, 4,000 test rows	±0.5481	0.57%
Soft tree (94.45%)	Binomial noise, 10,000 test images	±0.4579	0.48%
Tang (0.87%)	Binomial noise, 10,000 test images	±0.1857	21%
Wijaya (3.02), 1 run	None — reporting precision only	±0.005	0.17%
Wijaya (3.02), 3 seeds	Measured spread of 3 seeds	±0.2044	6.8%

- `inconclusive` — worse than the claim, but with no uncertainty estimate at all. The system asks for more seeds instead of ruling. This fired on Wijaya and changed the outcome.
- `not_evaluated` — nothing comparable: the run did not succeed, produced no value, stopped short of `full`, or targeted a claim the Reader does not have.

Table 8 is the honesty column of this report. A `pass` at ±0.00175 on 0.873 and a `pass` at ±0.2044 on 3.02 are not the same statement, and a bare verdict hides the difference. So the tolerance is printed everywhere the verdict appears, and when the band exceeds 10% of the claim the report adds an explicit warning. An earlier escalation of the same Wijaya claim passed at a mean of 3.3292 with a tolerance of ±1.0680 — 35% of the claim — and its report says in as many words that the result is *consistent with* the paper rather than a close match. Tang's ±0.1857 on a claim of 0.87% is the same situation in a different guise: a test error that small is simply not measurable to better than about a fifth of itself on 10,000 images.

5 The Three Experiments

Beyond the per-paper fidelity runs, three experiments were run to measure the system rather than the papers. Table 9 summarises them. Across all the runs this project holds, 32 carry a Critic verdict: 24 `pass`, 3 `inconclusive` and 5 `not_evaluated` — three cheap-stage runs, one generation that crashed at `probe`, and one ablation leg that never reached `full`.

5.1 Does the loop earn its place?

The project plan calls this the headline ablation, and the question is narrow: does the Coder–Runner–Critic loop improve replication over a single generation? Two arms, the same five papers, the same pinned claims, the same arithmetic judge. The loop-off arm runs with `--retry-budget 0 --no-critic`.

The short answer is that the loop changed the outcome on 1 of 5 papers, and on the other four it changed nothing. Single-shot reproduced 4 of 5 claims; loop-on reproduced 5 of 5.

That is a thin result and it should not be dressed up. Four of five papers paid for the loop — a Critic call each, plus the seed runs it is entitled to — and got the verdict single-shot

Table 9: The four measurements in this report, and what each one actually establishes.

Experiment	Design	Runs	Headline finding
Fidelity	5 papers, one `full` run each	5	5/5 pass; 2 better than the paper's own number
Ablation	5 papers × 2 arms, loop on and off	7	The loop changed 1 of 5 verdicts; the guards, not the retries, are the real defence
Consistency	2 papers × 5 independent generations	10	The SVM guide's generations agree to three decimals; Wijaya swings 2.6 to 4.0 with the claim inside the spread; 1 generation in 5 was born broken
End to end	3 papers × 3 runs, PDF to report, full loop	9	9/9 pass, every run `full` on the first attempt

Table 10: Loop off against loop on, five papers, same pinned claim and same judge. Wall clock is the `full` stage itself.

Paper	Claim	Loop off	Loop on	Wall clock, off / on	What the loop changed
Tang (`c5`)	0.87% error	0.85% — `pass`	0.82% — `pass`	1140 s / 3223 s	Nothing. But see the variance note below
Wijaya (`c3`)	RMSE 3.02	2.5802 — `pass` ↑	2.8843 — `pass` ↑	135 s / 140 s	Nothing; the loop never retried
SVM guide (`c1`)	96.9%	96.625 — `pass`	96.625 — `pass`	89 s / 92 s	Nothing. The identical number to three decimals, from a script only 35% similar
Fashion-MNIST (`c17`)	0.873	0.8773 — `pass` ↑	0.87648 — `pass` ↑	148 s / 178 s	Nothing; both exceed the claim
Soft tree (`c1`)	94.45%	**no number** — `retry_budget_exhausted`, fidelity `not_evaluated`	95.11% — `pass` ↑	6 s (halted) / 4191 s	**Everything.** The single-shot script implemented the paper's misprinted loss literally, a check-up halted it at `probe` epoch 1, and with no budget the run ended there

had already reached. Two of the five loop-on numbers were also human-assisted (Tang's hand-changed C, the soft tree's hand-corrected loss), so the loop-on arm is better described as "the loop plus the human attention this project actually gave these papers".

What the loop did contribute, robustly, is the guard rails rather than the retries. Note what did *not* happen on the soft tree: the pipeline did not report 7.8% accuracy as a replication result. A live check-up killed the container 6.18 seconds in, with its reason recorded — the training loss had reached −0.8361, below the loss's own lower bound of zero — and the Critic then refused to judge a halted run, returning `not_evaluated`. Both of those fired with retries switched *off*. On this evidence, ReproBot's main defence against publishing a wrong number is its guards; the retry loop is a comparatively rare repair path.

And the variance is real. Tang was run twice in the loop-off arm with the identical prompt and pinned claim. The kept leg scored 0.85% and passed; the discarded leg — discarded for an unrelated infrastructure fault, and reported here because the disagreement is itself the

Table 11: Five independent generations per paper, loop off, each judged by the same arithmetic. The standard deviations are five-sample estimates — an order of magnitude, not a measurement.

Paper	Claim	Passes	Reproduced, 5 generations	sd
SVM guide (`c1`)	96.9%	**5/5**	96.625, 96.625, 96.625, 96.625, **96.95**	0.145
Wijaya (`c3`)	RMSE 3.02	**1/5**	2.619, 3.703, 3.709, 4.003, *(one generation produced no number)*	0.609

finding — scored 1.17% and *failed*, a gap of +0.30 against a band of ±0.1857. Two different scripts, either side of the tolerance. One single-shot generation per paper is therefore a noisy measurement, and a five-paper, one-generation-per-arm design cannot resolve differences this size. Settling the question needs $k \approx 5$ independent generations per paper per arm.

5.2 How repeatable is one generation?

That conclusion motivated the next experiment: two cheap papers, five independent generations each, nothing shared between runs, loop off throughout so that what is measured is the Coder's own variance rather than the loop's repair rate.

Three things follow. **The variance tracks how precisely the paper specifies itself** — four of five SVM generations produced the identical number to three decimals, while Wijaya's reproduced RMSE swings by 1.38 between generations. **A single run is a weak verdict**: Wijaya's 1/5 is not "ReproBot fails this paper four times in five", because the spread *contains* 3.02; what varies is which side of the tolerance band one draw lands on, which is exactly why the Critic asks for seeds instead of ruling. And **one generation in five was born broken**: a script that called `.numpy()` on something that was not a tensor and died at `probe` in 6.5 seconds. Triage classified it `recoverable_error` — a bug a retry could fix — and with the budget at zero it simply ended. That is the population the retry loop serves, and it is the honest companion to the ablation's "the loop changed one verdict in five".

A further caution on this table: five of the fifteen attempted runs were lost to infrastructure (four to API connection errors and timeouts, one to a container killed by the operating system) and were re-run. Those are excluded, because they say nothing about the system under test. The broken generation *is* included, because that failure was the system's own.

5.3 End to end, with everything switched on

The third experiment is the one a demonstration has to pass: start at the PDF, finish at a rendered report, three times per paper. Its results are Table 4 and Section 3.3; what it adds to the other two is that the whole chain ran unattended, including the Critic's own escalation. A sample of what the final stage emits per run — headline, claim table, learning curve, the gap with its tolerance, the review, every unstated guess, the attempt timeline and the commands to run it again — is kept at `docs/notes/end-to-end/sample-report/`.

6 The Two Paper Sets

ReproBot is evaluated against two deliberately different sets of papers, and the distinction matters more than any single number in this report.

6.1 The CPU set: where the fidelity numbers come from

The original evaluation target was image classification on CIFAR-10. Full training of those papers is out of reach on this project's hardware — one *Wide Residual Networks* run at the paper's own setup measures at about 22 days on the development CPU — so a second set was assembled: papers whose headline experiment trains in seconds to about an hour on a CPU. This set exists for one reason. **It is the only way to obtain a measured fidelity number without GPU compute.** It was also chosen for model-family diversity, so the pipeline is tested on more than convolutional networks: Table 12 spans kernel machines, forests, boosting, trees trained by gradient descent, graph and text models, anomaly detection and tabular regression.

Three of the fifteen are blocked on one concrete thing: the Runner's Docker image has no `xgboost`, `lightgbm` or `catboost`. The rest are blocked only on transcription and compute time, both of which are cheap for this set.

6.2 The GPU set: eight CIFAR-10 papers, and no fidelity claim

The CIFAR-10 set in `dataset/` is the original evaluation target from the project proposal, and it is curated separately from the pipeline work by another member of the team — it is input to ReproBot, not a product of it. Table 13 states exactly how far each paper has got.

Two of the eight — Network In Network and Wide Residual Networks — were carried through the whole orchestrated loop to the `smoke` stage, both succeeding on the first attempt with no retry. That is worth stating precisely, because it is easy to over-read: **a one-epoch run over a few hundred images establishes that a generated script executes, and nothing whatsoever about whether its number is right.** No replication-fidelity claim exists for any paper in this set, and none can exist until the project has GPU compute. The reason is arithmetic, not pessimism: the `full` *Wide Residual Networks* configuration measures at about 22 days on the development CPU, against about a minute and a half to seventy minutes for each paper in the CPU set.

7 Problems and Solutions

Almost every design decision in the system traces to something that went wrong first. Table 14 lists the engineering problems and the fix each one forced; Table 15 lists the problems that showed up in *measuring* the system, which are a different kind of trap and easier to miss.

8 Honest Limits

The following are the limits of what this report establishes. They are stated plainly because a replication system that oversells itself is worse than useless.

- **One claim per run.** Fashion-MNIST has 74 extracted claims and one was tested. Every generated report lists the untested remainder as `not tested`, so the denominator is on the page — but nine end-to-end runs are three claims, not three papers replicated.
- **No GPU results at all**, and therefore no claim of any kind about the CIFAR-10 set the project originally set out to evaluate.
- **All five replicated papers were chosen because they reproduce on CPU in minutes.** A 9-of-9 pass rate is a statement about the pipeline running unattended on tractable papers, not a success rate on an arbitrary paper.

Table 12: The CPU set, `extra-papers/`, 15 papers. Target claims and data sources were read from each PDF directly and every dataset URL was checked to respond. Five have a `full` verdict; the rest are selected and documented but not yet transcribed.

Paper	Model family	Data	Target claim	Status
Breiman (2001) [1]	Random forest	13 small tabular sets, all on OpenML	Table 2 test error per dataset, e.g. breast cancer 2.9%, glass 20.6% — 13 claims from one paper	Selected; not yet transcribed
Pang, Lee & Vaithyanathan (2002) [2]	Naive Bayes / MaxEnt / linear SVM	1,400 movie reviews	SVM 82.9% with presence-weighted unigrams, three-fold CV	Selected; not yet transcribed
Hsu, Chang & Lin (2003) [3]	RBF SVM	svmguide1, 3,089/4,000	96.9% accuracy	**`full`: pass, 3/3 runs**
Liu, Ting & Zhou (2008) [5]	Isolation forest, unsupervised	9 sets, OpenML plus scikit-learn fetchers	Table 2 AUC, e.g. Shuttle 1.00, Pima 0.67 — the first target with no train/test split	Selected; not yet transcribed
Tang (2013) [8]	MLP with an L2-SVM loss	MNIST	0.87% test error	**`full`: pass**
Kim (2014) [10]	Text CNN	TREC	CNN-rand 91.2% accuracy	Selected; not yet transcribed
Chen & Guestrin (2016) [15]	Gradient boosting	HIGGS, 1M-row subset	test AUC 0.8304	Needs `xgboost` in the Runner image
Joulin et al. (2016) [16]	Linear bag-of-n-grams	AG News, 120k/7.6k	92.5% accuracy, and a claimed 1 s per epoch	Selected; not yet transcribed
Kipf & Welling (2016) [17]	Graph convolutional network	Cora	81.5% accuracy, mean of 100 initialisations	Selected; not yet transcribed
Prokhorenkova et al. (2017) [21]	CatBoost / LightGBM / XGBoost	Adult, OpenML 1590	log loss 0.2695, zero-one loss 0.1267	Needs three boosting libraries in the image
Xiao, Rasul & Vollgraf (2017) [19]	Random forest, in a classical zoo	Fashion-MNIST	0.873 accuracy, mean of 5 runs	**`full`: pass, exceeds**
Frosst & Hinton (2017) [20]	Soft decision tree	MNIST	94.45% accuracy	**`full`: pass, exceeds**
Bai, Kolter & Koltun (2018) [22]	Temporal convolutional network	Synthetic adding problem, $T = 600$	test MSE 5.8×10^{-5}	Selected; not yet transcribed
Bentéjac, Csörgő & Martínez-Muñoz (2019) [24]	XGBoost / random forest / gradient boosting	28 small tabular sets	Spambase, default XGBoost 95.17% ± 1.29	Needs `xgboost` in the image
Wijaya (2023) [26]	Dense regression network	Boston Housing, 405/101	test RMSE 3.02	**`full`: pass, 3/3 runs**

Table 13: The GPU set, `dataset/`, 8 CIFAR-10 papers, curated externally to the pipeline work. ✓ done; ✗ not done. No paper in this set has a fidelity verdict, and none can have one without GPU compute.

Paper	Date	OCR	Reader	Loop	Fidelity
Network In Network [9]	2013-12	✓	✓	`smoke`, success, 0 retries	`not_evaluated`
Striving for Simplicity: The All Convolutional Net [11]	2014-12	✓	✓	—	`not_evaluated`
Deep Residual Learning for Image Recognition [12]	2015-12	✓	✓	—	`not_evaluated`
Deep Networks with Stochastic Depth [13]	2016-03	✗	✗	—	`not_evaluated`
Wide Residual Networks [14]	2016-05	✓	✓	`smoke`, success, 0 retries	`not_evaluated`
Densely Connected Convolutional Networks [18]	2016-08	✗	✗	—	`not_evaluated`
AutoAugment: Learning Augmentation Policies from Data [23]	2018-05	✓	✗	—	`not_evaluated`
Escaping the Big Data Paradigm with Compact Transformers [25]	2021-04	✓	✗	—	`not_evaluated`
Total	8	**6**	**4**	**2**	**0**

- **Two of the five fidelity numbers were human-assisted** — one unstated hyperparameter changed after an ablation, one misprinted loss corrected — and both are marked as such wherever they appear.
- **The ablation is too thin to settle its own question**: five papers, one generation per arm, with two loop-on numbers human-assisted and three loop-on states captured on a zero-retry day. Three runs is enough to see whether a paper varies, not to estimate how much.
- **Re-runs overwrite `state.json`**, so a previous run's evidence is lost unless it was written to a separate output directory.
- **Retries regenerate the whole script** instead of patching the faulty lines, which makes each retry more expensive and less targeted than it needs to be.
- **Review ran on one generation per paper** in the end-to-end batch, so a deviation present only in the other two generations would not have been caught by the reviewer.
- **There is no demonstration interface yet**: a Streamlit dashboard over the pipeline's outputs exists on a feature branch, unmerged and unreviewed.
- **One dataset carries an ethical caveat.** Boston Housing includes a feature derived from the proportion of Black residents per town [6]; it is used here only because the replicated paper used it.

9 What Is Next

Patch, do not regenerate. A retry currently rewrites the whole script, which is why a regeneration can change an unstated hyperparameter that had nothing to do with the failure. The loop should edit the faulty lines and hold every other disclosed guess fixed unless the feedback concerns it.

Table 14: Problems the pipeline surfaced, and the fix each one forced. Most of these were found by running the thing, not by reading it.

Problem	Solution
HuggingFace's `Trainer` silently applied gradient clipping that no paper states, and the faithful version without it behaved differently	An explicit PyTorch training loop; every setting visible in the generated code
Re-implementing a classical model in PyTorch would change its solver and so its solution	The library is chosen by model family: scikit-learn for kernel machines, trees and forests, with paper-era version defaults set explicitly
A dataset URL recalled from memory had gone dead (HTTP 403) and triage filed it as a container fault, so the loop stopped with zero retries	OpenML first, pinned by `data_id`; a single refusing source is a fixable script error, not an environment error
A triage model confidently recalled Boston Housing as OpenML `data_id=506`, which is a different dataset	No identifier may be suggested unless it appears in the log; every generated script asserts the dataset's name and row/column counts before training
A regenerated script pointed `--data-dir` elsewhere and silently re-downloaded CIFAR-10: the same stage took 2000 s instead of 169 s, with no error	`--data-dir` must default to exactly `./data`, which the Runner bind-mounts; it is now part of the enforced command-line contract
`subprocess.TimeoutExpired` kills the `docker run` client while the container keeps running inside the daemon	Every run gets an explicit `--name` and a matching `docker kill`; partial output is decoded defensively, because the timeout attaches it as raw bytes even in text mode
A 1200 s probe budget would have timed out on the very first run — indistinguishable from a hung script	Timeouts calibrated from a measured run: a cold CIFAR-10 cache was 1,707 s of a 1,774 s probe, so `probe` carries the largest budget at 2700 s
A training run collapsed into a dead network at about 89% error and no stage noticed: the script exited 0	Live check-ups — seven pure rules reading the learning curve while the container runs, able to kill it by name and end the stage `halted` with evidence
A check-up false-positived and halted a correct soft tree in `capped`, whose loss sat at $\ln 10$ while its accuracy was three times chance	The `no_progress` rule restricted to `full` and required to show no gain over chance; the case kept as a replay test
The soft-tree paper prints its loss as the logarithm of a quantity that is never positive, so the faithful implementation maximises cross-entropy	Implemented as its evident intent once by hand, then the prompt was corrected. The single-shot arm later reproduced the same defect independently — and a check-up halted it in 6.18 s
A tool-use response returned its whole payload as a JSON string under one key on a clean `stop_reason`, so an extractor silently reported zero datasets	Handled centrally in one module, together with `max_tokens` truncation and all-empty payloads — three failure modes that each produce a valid but empty result
No PyTorch release supports Python 3.13 *and* ships Intel-macOS wheels, so a generated training script cannot run natively on the development machine at all	The Runner executes every script in Docker, where the container's own Linux Python makes the host's irrelevant. This is the platform trap's solution, not another instance of it

Table 15: Problems in measuring the system, and what was done about them. These bite quietly: each one produces a number that looks fine.

Problem	Solution
Target-claim selection is not reproducible: the same prompt picked a different Fashion-MNIST claim on two different days, so the two arms of a comparison would not have been comparing the same thing	Every experiment pins its target with `--claim-id`, read from the run it is being compared against
Re-running a paper overwrites its `state.json`, and one `full` run's evidence was lost that way to a later `capped` rerun	Every experiment writes to its own output directory. The underlying overwrite is still an open gap
A repo-wide formatter reformatted already-executed generated scripts in place, so the archived script no longer matched the text that ran	The difference is formatting only and no number is affected, but it is recorded per paper, and generated output is gitignored
Two single-shot generations of the same paper landed either side of the tolerance band, 0.85% against 1.17%	Reported as a finding rather than hidden; it is the direct reason the consistency experiment exists
A reviewer model that misses a bug does not stay silent — it misdiagnoses, and the loop then acts on the misdiagnosis	The review model was selected by a controlled test on a script with a planted bug; every citation is checked to exist, and a fix may not add a technique the paper never mentions

More than one claim per paper. The coverage table is the clearest gap in this report. Breiman alone offers 13 independent per-dataset claims and Isolation Forest nine, which would multiply the evaluation without multiplying the compute.

The boosting libraries. Three selected papers are blocked on `xgboost`, `lightgbm` and `catboost` being absent from the Runner image — the cheapest remaining widening of model-family coverage.

GPU compute for the CIFAR-10 set. Everything else about that set is ready: the loop carries its papers to `smoke` without a retry. What is missing is hardware, and until it arrives the set stays at `not_evaluated`.

Keep every run's evidence. Making each run's state immutable, rather than overwriting it, is a small change that would have preserved two results this project has already lost.

A demonstration. The Streamlit viewer should display Runner, Critic and report output — the curves and comparisons of this report — and merge.

10 Conclusion

ReproBot carries a paper from PDF to a judged replication report without supervision. Five papers across five model families reproduced their published numbers at the papers' own settings, two of them beating the published figure; three were carried end to end three times each and passed 9 times out of 9. The verdict behind each of those passes is arithmetic over the claim's own precision and the run's own measured spread, printed with its tolerance so that a wide pass cannot masquerade as a close match.

The result we did not expect is the one we would keep. When runs of the same paper disagreed, the disagreement was never mysterious: it was one sentence the paper did not write. A paper that states its protocol completely gave three unrelated scripts bit-identical numbers; a paper that omits whether its cross-validation folds are shuffled moved its own selected hyperparameters and its test accuracy; a paper that gives split sizes but not split assignment moved its RMSE continuously across a range that contains its claim. Reproducing a result automatically

turns out to be a good way of finding out precisely what a paper forgot to say — and the system's most valuable habit is not its retry loop but its refusal to report a number it cannot stand behind.

References

[1] Breiman, L. (2001). Random Forests. *Machine Learning*, 45(1), 5–32.

[2] Pang, B., Lee, L., & Vaithyanathan, S. (2002). Thumbs up? Sentiment Classification using Machine Learning Techniques. *Proceedings of EMNLP 2002*, 79–86.

[3] Hsu, C.-W., Chang, C.-C., & Lin, C.-J. (2003). A Practical Guide to Support Vector Classification. Technical report, Department of Computer Science, National Taiwan University.

[4] Chang, C.-C., & Lin, C.-J. (2011). LIBSVM: A library for support vector machines. *ACM Transactions on Intelligent Systems and Technology*, 2(3), 27.

[5] Liu, F. T., Ting, K. M., & Zhou, Z.-H. (2008). Isolation Forest. *Proceedings of the 8th IEEE International Conference on Data Mining (ICDM)*, 413–422.

[6] Harrison, D., & Rubinfeld, D. L. (1978). Hedonic housing prices and the demand for clean air. *Journal of Environmental Economics and Management*, 5(1), 81–102.

[7] Pedregosa, F., Varoquaux, G., Gramfort, A., et al. (2011). Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.

[8] Tang, Y. (2013). Deep Learning using Linear Support Vector Machines. *ICML 2013 Workshop on Challenges in Representation Learning*. arXiv:1306.0239.

[9] Lin, M., Chen, Q., & Yan, S. (2014). Network In Network. *arXiv:1312.4400*.

[10] Kim, Y. (2014). Convolutional Neural Networks for Sentence Classification. *Proceedings of EMNLP 2014*. arXiv:1408.5882.

[11] Springenberg, J. T., Dosovitskiy, A., Brox, T., & Riedmiller, M. (2015). Striving for Simplicity: The All Convolutional Net. *arXiv:1412.6806*.

[12] He, K., Zhang, X., Ren, S., & Sun, J. (2016). Deep Residual Learning for Image Recognition. *Proceedings of CVPR 2016*. arXiv:1512.03385.

[13] Huang, G., Sun, Y., Liu, Z., Sedra, D., & Weinberger, K. Q. (2016). Deep Networks with Stochastic Depth. *Proceedings of ECCV 2016*. arXiv:1603.09382.

[14] Zagoruyko, S., & Komodakis, N. (2016). Wide Residual Networks. *Proceedings of BMVC 2016*. arXiv:1605.07146.

[15] Chen, T., & Guestrin, C. (2016). XGBoost: A Scalable Tree Boosting System. *Proceedings of KDD 2016*. arXiv:1603.02754.

[16] Joulin, A., Grave, E., Bojanowski, P., & Mikolov, T. (2016). Bag of Tricks for Efficient Text Classification. *arXiv:1607.01759*.

[17] Kipf, T. N., & Welling, M. (2017). Semi-Supervised Classification with Graph Convolutional Networks. *Proceedings of ICLR 2017*. arXiv:1609.02907.

[18] Huang, G., Liu, Z., van der Maaten, L., & Weinberger, K. Q. (2017). Densely Connected Convolutional Networks. *Proceedings of CVPR 2017*. arXiv:1608.06993.

[19] Xiao, H., Rasul, K., & Vollgraf, R. (2017). Fashion-MNIST: a Novel Image Dataset for Benchmarking Machine Learning Algorithms. *arXiv:1708.07747*.

[20] Frosst, N., & Hinton, G. (2017). Distilling a Neural Network Into a Soft Decision Tree. *arXiv:1711.09784*.

[21] Prokhorenkova, L., Gusev, G., Vorobev, A., Dorogush, A. V., & Gulin, A. (2018). CatBoost: unbiased boosting with categorical features. *Advances in Neural Information Processing Systems 31*. arXiv:1706.09516.

[22] Bai, S., Kolter, J. Z., & Koltun, V. (2018). An Empirical Evaluation of Generic Convolutional and Recurrent Networks for Sequence Modeling. *arXiv:1803.01271*.

[23] Cubuk, E. D., Zoph, B., Mane, D., Vasudevan, V., & Le, Q. V. (2019). AutoAugment: Learning Augmentation Policies from Data. *Proceedings of CVPR 2019*. arXiv:1805.09501.

[24] Bentéjac, C., Csörgő, A., & Martínez-Muñoz, G. (2021). A comparative analysis of gradient boosting algorithms. *Artificial Intelligence Review*, 54, 1937–1967. arXiv:1911.01914.

[25] Hassani, A., Walton, S., Shah, N., Abuduweili, A., Li, J., & Shi, H. (2021). Escaping the Big Data Paradigm with Compact Transformers. *arXiv:2104.05704*.

[26] Wijaya, R. (2023). Multi Level Dense Layer Neural Network Model for Housing Price Prediction. *arXiv:2310.08133*.

[27] Huang, Q., Vora, J., Liang, P., & Leskovec, J. (2023). MLAgentBench: Evaluating Language Agents on Machine Learning Experimentation. *arXiv:2310.03302*.

[28] Starace, G., et al. (2025). PaperBench: Evaluating AI's Ability to Replicate AI Research. OpenAI. *arXiv:2504.01848*.

[29] Seo, M., et al. (2025). Paper2Code: Automating Code Generation from Scientific Papers in Machine Learning. *arXiv:2504.17192*.

[30] Lin, Z., et al. (2025). AutoP2C: An LLM-Based Agent Framework for Code Repository Generation from Multimodal Content in Academic Papers. *arXiv:2504.20115*.

[31] Zhao, X., Sun, M., Wang, W., et al. (2025). AutoReproduce: Automatic AI Experiment Reproduction with Paper Lineage. *arXiv:2505.20662*.